Практика 5: Операционные компоненты + OLAP

Table of contents

Практика 5: Операционные компоненты + OLAP 1
Предисловие 1
Задание 1
Пример 2

Практика 5: Операционные компоненты + OLAP

Предисловие

Написать работающий код — только половина задачи. Продакшн-система требует операционных компонентов: защита от перегрузки (ограничение частоты запросов), распределение нагрузки (балансировка нагрузки), безопасность (аутентификация/авторизация), наблюдаемость (метрики, логи, трейсы). Также большинство систем нуждаются в аналитике — а это требует отдельного OLAP-хранилища и ETL-пайплайнов.

Задание

Дополните вашу OLTP-систему из Практики 3 **операционными компонентами**:

Часть 1: Ограничение частоты запросов и балансировка нагрузки

1. **Ограничение частоты запросов:**
   - Выберите алгоритм: Token Bucket, Sliding Window Log, Sliding Window Counter
   - Определите лимиты: для авторизованных и неавторизованных пользователей
   - Обоснуйте выбор
2. **Балансировка нагрузки:**
   - Предположим, у вас 3 инстанса приложения
   - Выберите алгоритм: Round Robin, Least Connections, IP Hash, Consistent Hashing
   - Обоснуйте выбор

Часть 2: Безопасность

1. **Аутентификация:**
   - Выберите механизм: JWT, сессии, OAuth 2.0
   - Опишите последовательность запросов
2. **Авторизация:**
   - Выберите модель: RBAC, ABAC
   - Определите роли/права для вашего сценария

Часть 3: Наблюдаемость

1. **Метрики (Prometheus):**

- Определите 5-7 ключевых метрик для мониторинга
- Укажите тип: Counter, Gauge, Histogram, Summary
- Напишите примеры PromQL-запросов

2. **Логирование:**
   - Определите формат: структурированный JSON
   - Какие поля включать (timestamp, level, user_id, trace_id, etc.)
3. **Трассировка:**
   - Опишите span для типичного запроса (например, создание заказа)

Часть 4: OLAP и аналитика

1. **Требования к аналитике:**
   - Какие отчёты нужны? (например: топ-товары, динамика продаж, RFM-анализ клиентов)
2. **ETL-конвейер:**
   - Как передавать данные из OLTP (PostgreSQL) в OLAP (ClickHouse)?
   - Выберите инструмент: Airflow, dbt, собственный скрипт
   - Опишите периодичность (в реальном времени, раз в час, раз в день?)
3. **Расчёт объёма:**
   - Сколько данных в день передаётся в OLAP?
   - Формат: Parquet, CSV, JSON?
   - Компрессия: gzip, zstd?

Формат ответа: документ с архитектурной схемой и обоснованиями.

Пример

Часть 1: Ограничение частоты запросов и балансировка нагрузки

1. Ограничение частоты запросов

Алгоритм: Token Bucket

Лимиты:

```
- Неавторизованные пользователи: 10 req/min (защита от парсеров)
- Авторизованные пользователи: 100 req/min
- API для мобильного приложения: 200 req/min
- Админы: без лимита
```

Конфигурация (nginx):

```
http {
    # Зона для ограничения частоты запросов
    limit_req_zone $binary_remote_addr zone=general:10m rate=10r/m;
    limit_req_zone $user_id zone=authenticated:10m rate=100r/m;
```

```
    server {
        location /api/ {
            # Неавторизованные
            limit_req zone=general burst=20;

            # Если есть токен — используем зону authenticated
            if ($http_authorization) {
                limit_req zone=authenticated burst=50;
            }
        }
    }
}
```

Обоснование Token Bucket:

- Позволяет кратковременные всплески: пользователь может сделать 20 быстрых запросов, потом ограничение
- Простая реализация в nginx/Redis
- Sliding Window Log был бы точнее, но избыточен

2. Балансировка нагрузки

Алгоритм: Least Connections

Конфигурация (nginx):

```
upstream backend {
    least_conn;  # выбирает сервер с наименьшим числом активных соединений

    server app1.internal:3000;
    server app2.internal:3000;
    server app3.internal:3000;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}
```

Обоснование:

- Запросы неравномерные: создание заказа (долгий) vs просмотр товара (быстрый)
- Least Connections автоматически учитывает загруженность
- Round Robin не учитывает нагрузку — может перегрузить один сервер

- IP Hash не нужен (нет привязки сессии к серверу)

Часть 2: Security

1. Аутентификация: JWT

Последовательность запросов:

```
1. POST /auth/login {email, password}
   → Сервер проверяет пароль
   → Возвращает JWT: {user_id, roles, exp}

2. Клиент сохраняет JWT (localStorage/cookie)

3. Все последующие запросы:
   GET /api/cart
   Authorization: Bearer <JWT>

4. Сервер проверяет подпись JWT (без обращения к БД!)
```

JWT structure:

```
{
  "header": {"alg": "HS256", "typ": "JWT"},
  "payload": {
    "user_id": 12345,
    "email": "alice@example.com",
    "roles": ["customer"],
    "exp": 1704067200  // срок действия: 1 час
  },
  "signature": "..."
}
```

Обоснование:

- Без хранения состояния на сервере: не нужно держать сессии в Redis/БД
- Масштабируемость: любой инстанс может проверить JWT
- Модель на сессиях требует централизованное хранилище (Redis)

2. Авторизация: RBAC

Роли:

```
- customer: может просматривать товары, управлять корзиной, создавать заказы
- admin: может управлять товарами, просматривать все заказы
- support: может просматривать заказы, менять статусы
```

Права (permissions):

```go
const (
    ReadProducts    = "products:read"
    WriteProducts   = "products:write"
    ReadOrders      = "orders:read"
    WriteOrders     = "orders:write"
    ReadAllOrders   = "orders:read:all"  // свои vs все
    UpdateOrderStatus = "orders:update:status"
)

var roles = map[string][]string{
    "customer": {ReadProducts, WriteOrders, ReadOrders},
    "admin":    {ReadProducts, WriteProducts, ReadAllOrders, UpdateOrderStatus},
    "support":  {ReadProducts, ReadAllOrders, UpdateOrderStatus},
}
```

Middleware для проверки:

```go
func RequirePermission(perm string) middleware {
    return func(c *Context) {
        userRoles := c.Get("roles")  // из JWT
        if !hasPermission(userRoles, perm) {
            c.AbortWithStatus(403)  // Forbidden
        }
    }
}

// Использование
router.POST("/products", RequirePermission(WriteProducts), createProduct)
router.GET("/orders", RequirePermission(ReadAllOrders), listAllOrders)
```

Обоснование RBAC:

- Простая модель для типичного интернет-магазина
- Роли статичны (customer, admin)
- ABAC избыточен (не нужны динамические правила вроде “только в рабочее время”)

Часть 3: Observability

1. Метрики (Prometheus)

Ключевые метрики:

```go
import "github.com/prometheus/client_golang/prometheus"
```

```go
// 1. Counter: общее количество запросов
var httpRequestsTotal = prometheus.NewCounterVec(
    prometheus.CounterOpts{Name: "http_requests_total"},
    []string{"method", "endpoint", "status"},
)

// 2. Histogram: латентность запросов
var httpDuration = prometheus.NewHistogramVec(
    prometheus.HistogramOpts{
        Name: "http_request_duration_seconds",
        Buckets: []float64{.001, .01, .1, 1, 10},  // 1ms, 10ms, 100ms, 1s, 10s
    },
    []string{"method", "endpoint"},
)

// 3. Gauge: активные соединения
var activeConnections = prometheus.NewGauge(
    prometheus.GaugeOpts{Name: "http_active_connections"},
)

// 4. Counter: заказы
var ordersCreated = prometheus.NewCounter(
    prometheus.CounterOpts{Name: "orders_created_total"},
)

// 5. Counter: ошибки БД
var dbErrors = prometheus.NewCounterVec(
    prometheus.CounterOpts{Name: "db_errors_total"},
    []string{"operation"},  // select, insert, update
)

// 6. Histogram: размер корзины
var cartSize = prometheus.NewHistogram(
    prometheus.HistogramOpts{
        Name: "cart_items_count",
        Buckets: []float64{1, 3, 5, 10, 20},
    },
)

// 7. Gauge: остаток товаров на складе
var productStock = prometheus.NewGaugeVec(
    prometheus.GaugeOpts{Name: "product_stock"},
```

```
    []string{"product_id"},
)
```

PromQL запросы:

```
# 1. RPS (запросов в секунду)
rate(http_requests_total[1m])

# 2. Error rate (процент 5xx ошибок)
sum(rate(http_requests_total{status=~"5.."}[1m]))
/
sum(rate(http_requests_total[1m]))

# 3. p95 латентность
histogram_quantile(0.95,
  rate(http_request_duration_seconds_bucket[5m])
)

# 4. Количество заказов за последний час
increase(orders_created_total[1h])

# 5. Топ медленных эндпоинтов
topk(5,
  histogram_quantile(0.99,
    rate(http_request_duration_seconds_bucket[5m])
  ) by (endpoint)
)
```

Grafana дашборд (панели):

- RPS по эндпоинтам (линейный график)
- Error rate (линейный график с алертом > 1%)
- Латентность p50/p95/p99 (линейный график)
- Активные соединения (gauge)
- Количество заказов за день (counter)

2. Логирование

Формат: Structured JSON

```
{
  "timestamp": "2025-01-02T14:30:15.123Z",
  "level": "INFO",
  "service": "ecommerce-api",
```

```
  "trace_id": "a1b2c3d4-e5f6-7890",
  "span_id": "f1e2d3c4",
  "user_id": 12345,
  "method": "POST",
  "endpoint": "/api/orders",
  "status": 201,
  "duration_ms": 45,
  "message": "Order created successfully",
  "order_id": 98765
}
```

Пример ошибки:

```
{
  "timestamp": "2025-01-02T14:32:10.456Z",
  "level": "ERROR",
  "service": "ecommerce-api",
  "trace_id": "a1b2c3d4-e5f6-7890",
  "span_id": "f1e2d3c4",
  "user_id": 12345,
  "method": "POST",
  "endpoint": "/api/orders",
  "error": "insufficient stock",
  "product_id": 555,
  "requested": 10,
  "available": 3,
  "message": "Failed to create order"
}
```

Преимущества:

- Легко фильтровать по полям (Elasticsearch, Loki)
- Trace_id связывает логи одного запроса
- Можно парсить и строить метрики

3. Distributed Tracing (OpenTelemetry + Jaeger)

Span для создания заказа:

```
Trace: "POST /api/orders" (trace_id: abc-def)
├─ Span 1: "http.request" (45 мс total)
│  ├─ Span 2: "validate_cart" (2 мс)
│  │  └─ Span 3: "redis.get cart:12345" (0.5 мс)
│  ├─ Span 4: "db.transaction.create_order" (40 мс)
```

```
│  │  ├─ Span 5: "db.insert orders" (10 мс)
│  │  ├─ Span 6: "db.insert order_items" (5 мс)
│  │  └─ Span 7: "db.update products.stock" (15 мс)  ← медленный!
│  └─ Span 8: "send_email_notification" (3 мс async)
```

Вывод из трейса: обновление stock занимает 15 мс (самая медленная операция). Возможная оптимизация: батчинг UPDATE или использовать Redis для быстрой проверки остатков.

Часть 4: OLAP и аналитика

1. Требования к аналитике

Отчёты для бизнеса:

- **Топ-товары** (самые продаваемые за день/неделю/месяц)
- **Динамика продаж** (revenue по дням, сравнение с прошлым периодом)
- **RFM-анализ клиентов** (Recency, Frequency, Monetary — сегментация)
- **Конверсия корзины** (% пользователей, совершивших заказ)
- **AOV** (Average Order Value — средний чек)

2. ETL-конвейер (Airflow)

Архитектура:

```
PostgreSQL (OLTP)
    ↓ (каждый час)
Airflow DAG: извлечение → преобразование → загрузка
    ↓
ClickHouse (OLAP)
```

Airflow DAG:

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

dag = DAG(
    'oltp_to_olap_etl',
    start_date=datetime(2025, 1, 1),
    schedule_interval='@hourly',  # каждый час
)

def extract_orders():
    # Читаем новые заказы из PostgreSQL
    conn = psycopg2.connect("postgresql://...")
```

```
    df = pd.read_sql("""
        SELECT o.id, o.user_id, o.created_at, o.total_amount,
               oi.product_id, oi.quantity, oi.price
        FROM orders o
        JOIN order_items oi ON oi.order_id = o.id
        WHERE o.created_at >= NOW() - INTERVAL '1 hour'
    """, conn)
    df.to_parquet('/tmp/orders_hourly.parquet')

def transform_data():
    df = pd.read_parquet('/tmp/orders_hourly.parquet')
    # Добавляем метаданные
    df['extracted_at'] = datetime.now()
    df.to_parquet('/tmp/orders_transformed.parquet')

def load_to_clickhouse():
    df = pd.read_parquet('/tmp/orders_transformed.parquet')
    # Загружаем в ClickHouse
    client = clickhouse_connect.get_client(host='clickhouse.internal')
    client.insert_df('orders_fact', df)

extract_task                =                PythonOperator(task_id='extract',
python_callable=extract_orders, dag=dag)
transform_task              =              PythonOperator(task_id='transform',
python_callable=transform_data, dag=dag)
load_task = PythonOperator(task_id='load', python_callable=load_to_clickhouse,
dag=dag)

extract_task >> transform_task >> load_task
```

Периодичность:

- **Раз в час** для оперативных дашбордов (динамика продаж в реальном времени)
- **Раз в день** для тяжёлых агрегаций (RFM-анализ)

3. Расчёт объёма

Данные в день:

```
10,000 заказов/день × 2 товара/заказ = 20,000 строк
Размер строки: ~100 байт (order_id, user_id, product_id, quantity, price,
timestamps)
20,000 × 100 байт = 2 МБ/день
```

Формат: **Parquet**

- Колоночный формат (оптимален для ClickHouse)
- Встроенная компрессия (zstd)
- Эффективно для пакетной загрузки раз в час

Компрессия:

```
2 МБ (raw) → ~0.5 МБ (Parquet + zstd)
```

Объём за год:

```
0.5 МБ/день × 365 = 182 МБ/год
```

ClickHouse схема:

```
CREATE TABLE orders_fact (
    order_id UInt64,
    user_id UInt64,
    product_id UInt64,
    quantity UInt16,
    price Decimal(10, 2),
    created_at DateTime,
    extracted_at DateTime
)
ENGINE = MergeTree()
PARTITION BY toYYYYMM(created_at)  -- партиционирование по месяцам
ORDER BY (user_id, created_at);  -- сортировка для быстрых запросов по user_id
```

Пример запроса (топ-товары):

```
SELECT
    product_id,
    SUM(quantity) AS total_sold,
    SUM(quantity * price) AS revenue
FROM orders_fact
WHERE created_at >= today() - INTERVAL 7 DAY
GROUP BY product_id
ORDER BY revenue DESC
LIMIT 10;
```

Итого:

- OLTP ⟶ OLAP: раз в час через Airflow
- Формат: Parquet (колоночный + компрессия)
- Объём: ~0.5 МБ/день ⟶ 182 МБ/год (легко масштабируется)